



Proyecto Integrador: Diseño de Arquitectura Híbrida para Ecommify

Asignatura: Diseño y Optimización de Bases de Datos

Docente: Miguel Alfonso Varela Fonseca

Integrantes del Equipo:
Juan Daniel Valderrama Pérez
Jorge Esteban Triviño Correa
Javier Andres Baron Fontanilla

Resumen del documento

Proyecto integrador para el diseño de una arquitectura híbrida en Ecommify, con enfoque políglota para un escenario de comercio electrónico.

Puntos clave

Base híbrida: PostgreSQL para transacciones atómicas.

MongoDB para el catálogo de productos con atributos variables.

Objetivo: balancear consistencia, disponibilidad y flexibilidad.

Documento de referencia: diseño conceptual y restricciones de negocio.

Arquitectura elegida

PostgreSQL: órdenes, pagos, estados transaccionales.

MongoDB: catálogo de productos, reseñas y variabilidad por categoría.

El documento prioriza una solución híbrida basada en CAP.

Reglas de negocio

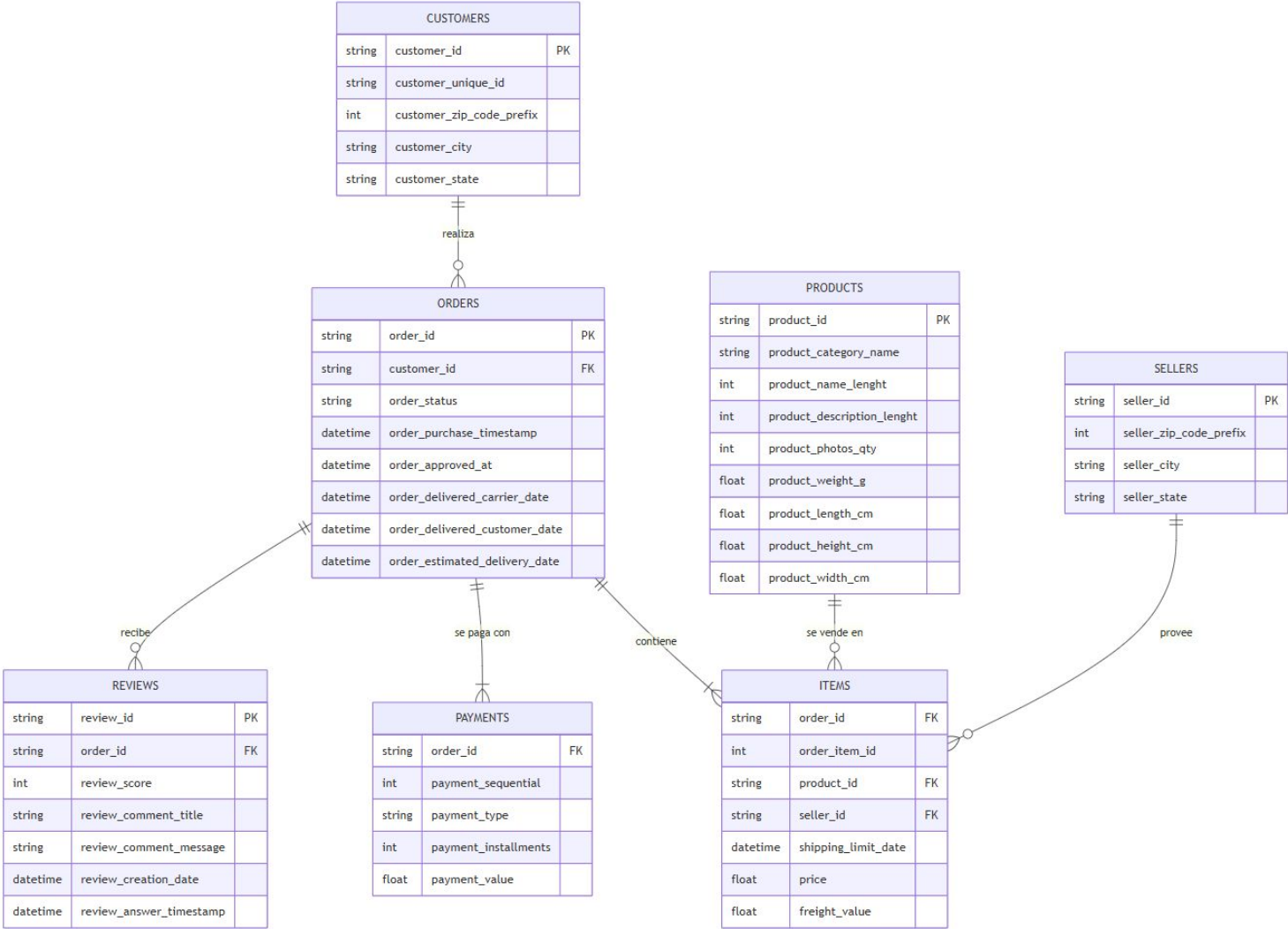
Pedidos y pagos

- Integridad Transaccional - Pedidos y Pagos
- Logística y Vendedores
- Calidad y Experiencia - Reviews
- Catálogo de Productos

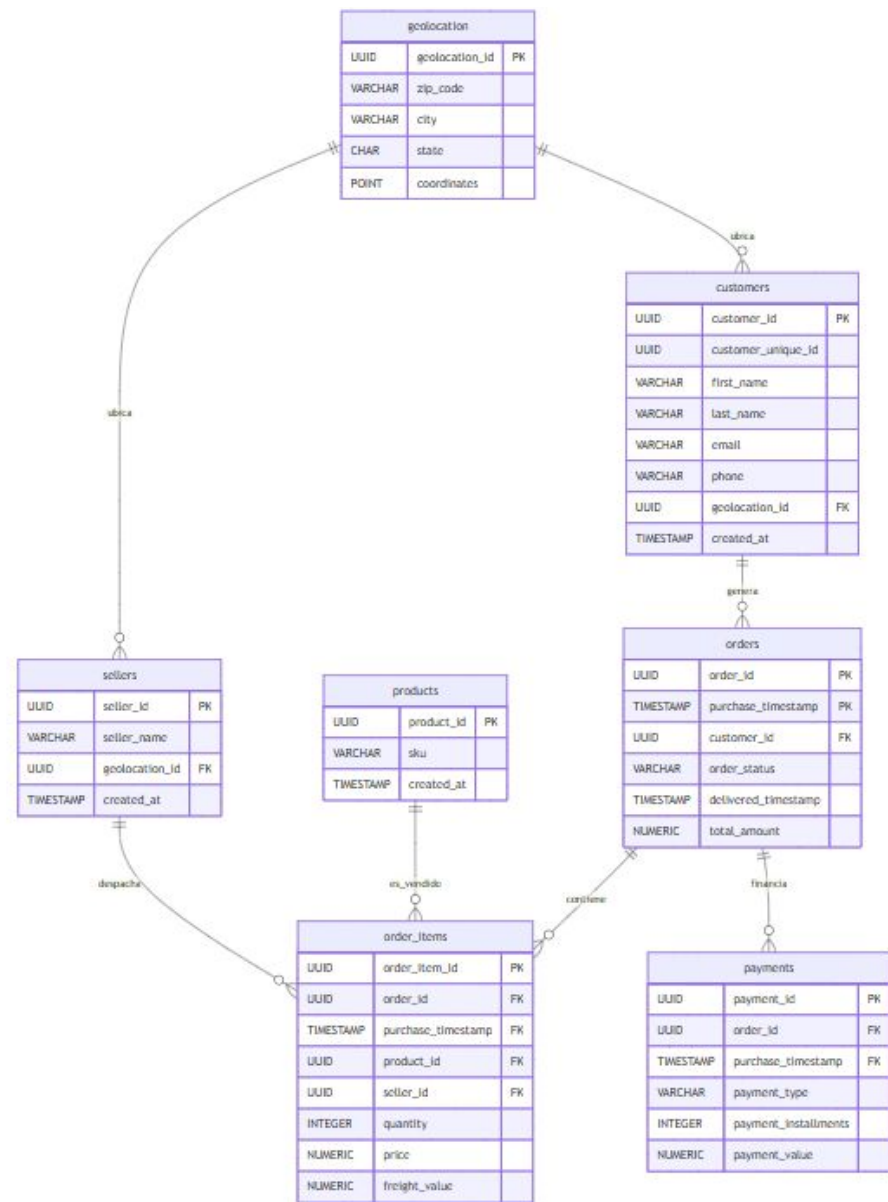
Logística, reseñas y catálogo

- customer/seller deben existir en geolocation por zip_code_prefix.
- Las reseñas solo aplican a órdenes entregadas o enviadas.
- Peso y dimensiones del producto deben ser mayores a cero.

Modelo ER Original del Dataset
Ollist Sin Normalizar



Modelo ER Propuesto Para el Módulo de PostgreSQL Normalizado 3FN



Tablas y claves principales

Tabla	Claves / atributos clave
customers	customer_id PK, customer_unique_id, zip_code_prefix, city, state
orders	order_id PK, purchase_timestamp FK, customer_id FK, order_status, timestamps
order_items	order_item_id PK, order_id FK, purchase_timestamp FK, product_id, seller_id, price, freight_value
payments	payment_id PK, order_id FK, purchase_timestamp FK, payment_type, installments, payment_value
geolocation	geolocation_id PK, zip_code, city, state, coordinates
products	product_id PK, category, name/description lengths, photos, dimensions
sellers	seller_id PK, zip_code_prefix, city, state

Relaciones 1:N entre customers-orders, orders-items, orders-payments, orders-reviews, products-items y sellers-items.

Estado actual

Campo	Tipo avanzado	Justificación
product_specifications	JSONB	Atributos variables de productos
product_images	TEXT[]	Manejo eficiente de múltiples URLs
promotion_period	TSTZRANGE	Gestión temporal de promociones
metadata	JSONB	Información flexible de auditoría
coordinates	POINT/PostGIS	Consultas geoespaciales

Evaluadas según las entidades

- **pg_trgm**: Búsqueda tolerante a errores
- **PostGIS**: Consultas geoespaciales
- **pgcrypto**: Generación segura de UUID
- **hstore**: Metadatos clave-valor simples

Colección preliminar

products_catalog

```
"properties": {  
  "_id": { "type": "string" },  
  "product_id_pg": { "type": "string" },  
  "category_name": { "type": "string" },  
  "name": { "type": "string" },  
  "description": { "type": "string" },  
  "images": {  
    "type": "array",  
    "items": { "type": "string" }  
  },  
  "attributes": {  
    "type": "array",
```

Idea de uso

El catálogo debe aceptar atributos variables.

MongoDB permite extender categorías sin cambiar el esquema global.

Se pueden agregar subdocumentos o arreglos por tipo de producto.

En el documento técnico se detallan los esquemas por colección.

Comparación porcentual de paradigma NoSQL y SQL

Criterio	Peso	PostgreSQL	MongoDB
Consistencia transaccional	40%	5	3
Flexibilidad del esquema	20%	2	5
Catálogo multivariable	20%	3	5
Operación de pagos/órdenes	20%	5	2
Puntaje ponderado	100%	4.2	3.8

En el documento técnico se detalla la decisión por entidad y su aplicación tanto en PostgreSQL como en MongoDB.

Lectura para la solución híbrida

Consistencia (C)

Prioridad en órdenes y pagos.
Evita inconsistencias monetarias.
Preferido para el núcleo transaccional.

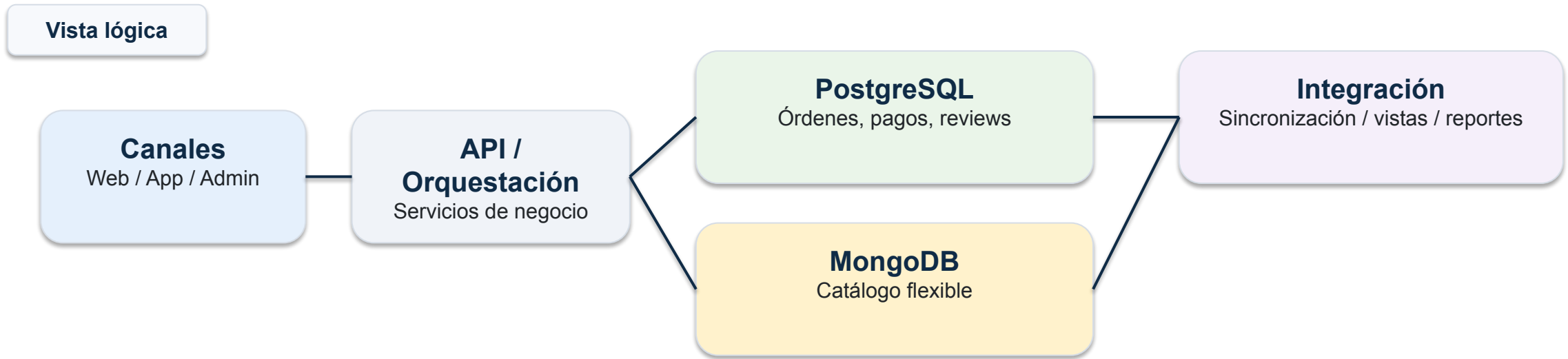
Disponibilidad (A)

Importante en el catálogo completo de productos y consulta.
Acepta mejor la lectura continua.
Útil en navegación de productos.

Tolerancia a particiones (P)

Soporta fallas parciales de red.
La arquitectura no depende de un único motor.
El balance cambia según el subsistema.

En el documento técnico se observa que la decisión se inclina a un balance: consistencia fuerte en transacciones y flexibilidad en el catálogo.



PostgreSQL soporta el core transaccional de mayor peso para el negocio.

MongoDB desacopla el catálogo para permitir atributos variables.

La capa de integración facilita consultas consolidadas y reportes.

Riesgos y respuestas

Limitaciones

- Doble persistencia y mayor complejidad operativa.
- Posible inconsistencia temporal entre motores.
- Consultas cruzadas más costosas.
- Necesidad de gobernanza de datos y monitoreo.

Mitigaciones

- Definir una fuente de verdad por dominio.
- Sincronizar catálogos con procesos controlados.
- Implementar validaciones y pruebas de consistencia.
- Agregar backups, observabilidad y alertas.

Hoja de ruta

1

Cierre del modelo conceptual y relacional.

2

Provisionamiento de PostgreSQL y MongoDB.

3

Implementación del núcleo transaccional.

4

Implementación del catálogo flexible.

5

Integración, pruebas, documentación y ajustes.

El plan es ejecutado según ampliaciones con cronograma, responsables y entregables.



GRACIAS